JAVA · SPRING BOOT · TEST ENGINEERING

Deep dive in practical software testing

A pattern guide for unit, integration, and end-to-end tests

Version 2

Java 21 · Spring Boot 4.1 · JUnit 5 · AssertJ
Mockito · Testcontainers · Awaitility · Playwright

Examples use a calculator, a word counter, and a payment service.
The same patterns apply to any Spring Boot API on this stack.

1. What the three levels prove

A test is evidence. It shows that a business rule is right, that two components collaborate across a boundary, or that one customer journey completes. A coverage percentage is a floor under that evidence. A test that executes a line and checks nothing raises the number and proves nothing.

The level is the boundary the test crosses. The annotation is only how you start that boundary. `@SpringBootTest` loads Spring. With `MockMvc` the call stays inside the process, and the test is integration. A real port makes it end to end: REST Assured reads the JSON, and Playwright opens the page.

	Unit	Integration	End to end
Proves	One rule	Components across a boundary	One full-stack journey
Spring	Absent	A slice, or the full context	The process, listening on a real port
Database	None, or a mock	The engine you ship, in a container	That same database, reached by HTTP or the UI
Speed	The whole unit set in seconds	A container start, then HTTP	Slowest. Keep the set small.
A failure looks like	A wrong total or a wrong exception	SQL, JSON names, security, a commit	A wrong JSON field, a filter, or a broken screen

Put each behaviour at the lowest level that can go red for that bug. A fee formula does not need PostgreSQL. A missing authorization header does not need Chromium. Add a wider test only where the boundary itself can be wrong.

2. Where each test sits

Constructor injection is the default shape of testable code. The service receives its repository and its clock. The test supplies a mock and a fixed clock. The method under test stays an ordinary instance method.

```
public class PaymentService {
    private final PaymentRepository payments;
    private final Clock clock;

    public PaymentService(PaymentRepository payments, Clock clock) {
        this.payments = payments;
        this.clock = clock;
    }
}
```

A pure function with no collaborator may be static. `FeeCalculator.total` is that kind of method. A service that calls a repository is an instance. Making it static so a test can call it without `new` removes the seam the mock needs.

```
[ Client request ]
        |
        v
+-------------------------------------------+
| Web / controller                          |
| @WebMvcTest + MockMvc           integration |
+-------------------------------------------+
        |
        v
+-------------------------------------------+
| Service                                   |
| MockitoExtension + AssertJ      unit       |
+-------------------------------------------+
        |
        v
+-------------------------------------------+
| Repository                                |
| @DataJpaTest + Testcontainers   integration |
+-------------------------------------------+
        |
        v
+-------------------------------------------+
| HTTP API                                  |
| REST Assured + RANDOM_PORT      end to end  |
+-------------------------------------------+
        |
        v
+-------------------------------------------+
| Browser journey                           |
| Playwright + RANDOM_PORT        end to end  |
+-------------------------------------------+
```

Read the diagram from the bug outward. If the bug is inside `FeeCalculator`, stop at the unit row. If the bug is a column type or a Flyway script, stop at the repository row. If the bug is a JSON field, stop at the HTTP row and assert it with JSONPath. If the bug is a button that never reaches the API, you are on the last row.

3. Names, and Arrange - Act - Assert

Every test has three blocks, in this order. Arrange prepares inputs and mocks. Act calls one method or sends one request. Assert checks the value, the exception, or the collaborator call.

The method name is the sentence a failure prints. Write three parts, separated by underscores: `subject_condition_outcome`. Use camelCase inside each part. Do not prefix the method with `test`.

Part	Names	Example
subject	The method, route, or type	add, count, charge, health
condition	The input or state that matters	twoAndThree, byZero, fiveItems
outcome	The result the assert checks	returnsFive, throws, hidesVersion

`add_twoAndThree_returnsFive` reads as: add, given 2 and 3, returns 5. One outcome per method. A name that joins two results with "and" is two methods. Two inputs that share one result may stay together when the name says so: `count_blankOrNull_returnsZero`.

Assertions in this guide use AssertJ, which ships with `spring-boot-starter-test`. AssertJ reads the actual value first.

```
assertThat(sum).isEqualTo(5);
assertThat(found).isNull();
assertThatThrownBy(() -> Calculator.divide(10, 0))
        .isInstanceOf(IllegalArgumentException.class)
        .hasMessage("divisor must not be zero");
```

4. Assert and verify

An assert and a verify both sit in the Assert block. They check different facts.

An **assert** looks at a value you hold after the call: the number, the exception, the message. It is AssertJ. The question is: what did this method produce?

A **verify** looks at a collaborator: which method ran, with which arguments, and how many times. It is Mockito. The question is: whom did this method talk to?

`Calculator.add` and `WordCounter.count` only produce a value. There is nothing to verify.

```
@Test
void add_twoAndThree_returnsFive() {
    // Arrange
    Calculator calculator = new Calculator();

    // Act
    int sum = calculator.add(2, 3);

    // Assert
    assertThat(sum).isEqualTo(5);
}
```

`./mvnw verify` is a Maven phase. It runs tests and the coverage gate. It is not Mockito's `verify`.

A method that returns a value and talks to someone

`countAndStore` counts words and, when the count is not zero, saves that count.

```
public class CountService {
    private final WordCounter counter;
    private final CountStore store;

    public CountService(WordCounter counter, CountStore store) {
        this.counter = counter;
        this.store = store;
    }

    public int countAndStore(String text) {
        int n = counter.count(text);
        if (n == 0) {
            return 0;
        }
        store.save(text, n);
        return n;
    }
}
```

```
@Test
void countAndStore_twoWords_returnsTwo() {
    // Arrange
    when(counter.count("hello world")).thenReturn(2);
    CountService service = new CountService(counter, store);

    // Act
    int n = service.countAndStore("hello world");

    // Assert the value, then the conversation
    assertThat(n).isEqualTo(2);
    verify(store, times(1)).save("hello world", 2);
}
```

Bug inside countAndStore	assertThat(n).isEqualTo(2)	verify save(text, 2)
Returns 2 and never calls save	Green	Red. The store was not called.
Saves 2 and returns 99	Red. The number is wrong.	Green. The save was right.
Saves 0 and returns 2	Green	Red. The saved number is wrong.

Write the check that matches the promise. Write both when both are promises. `times(1)` is Mockito's default; write it when "once" is the rule. `never()` requires that a call did not happen. `verifyNoInteractions` requires that no method on that mock ran. `when(...).thenReturn(...)` is Arrange. `verify` comes after the act.

5. Unit tests

5.1 Calculator and word count

```
public final class Calculator {
    public int add(int a, int b) { return a + b; }

    public int divide(int a, int b) {
        if (b == 0) {
            throw new IllegalArgumentException(
                    "divisor must not be zero");
        }
        return a / b;
    }
}
```

```
public final class WordCounter {
    public int count(String text) {
        if (text == null || text.isBlank()) {
            return 0;
        }
        return text.trim().split("\\s+").length;
    }
}
```

One sentence is one test. Adding 2 and 3 returns 5. Dividing by zero throws. Blank text counts as zero. Integer division truncates: `divide(5, 2)` returns 2, and that fact deserves its own test so nobody "fixes" it into a float by accident.

```
@Test
void divide_byZero_throwsIllegalArgument() {
    // Arrange
    Calculator calculator = new Calculator();

    // Act + Assert
    assertThatThrownBy(() -> calculator.divide(10, 0))
            .isInstanceOf(IllegalArgumentException.class)
            .hasMessage("divisor must not be zero");
}

@Test
void divide_tenByTwo_returnsFive() {
    // Arrange
    Calculator calculator = new Calculator();
    // Act
    int quotient = calculator.divide(10, 2);
    // Assert
    assertThat(quotient).isEqualTo(5);
}
```

An edge sits on the line where the result changes. Test both sides, and the line. For `count`, the edges are one word, the first space, extra spaces, `""`, blank, and `null`. `null` and double spaces fail for different bugs. A method that only calls `isEmpty` throws on `null` and counts the gaps as words.

5.2 A fee rule, still a unit test

The same shape carries a business rule. Ten percent off from five items. Below five, full price. Negative quantity is refused.

```
public final class FeeCalculator {
    public static long total(int quantity, long unitPrice) {
        if (quantity < 0 || unitPrice < 0) {
            throw new IllegalArgumentException(
                    "quantity and unitPrice must be non-negative");
        }
        long subtotal = quantity * unitPrice;
        return quantity >= 5 ? subtotal * 90 / 100 : subtotal;
    }
}
```

```
@Test
void total_fourItems_chargesFullPrice() {
    assertThat(FeeCalculator.total(4, 100)).isEqualTo(400);
}

@Test
void total_fiveItems_appliesTenPercent() {
    assertThat(FeeCalculator.total(5, 100)).isEqualTo(450);
}
```

Shipping can follow the same table habit. Standard shipping is 2,000 until the amount reaches 50,000, then 0. A gold customer ships free below that line as well. One row, one test, one name: `shipping_standardJustBelowThreshold_costs2000`, `shipping_standardAtThreshold_isFree`, `shipping_goldBelowThreshold_isFree`.

5.3 Static, private, and the clock

These three are awkward for different reasons. A static method is still a direct call. A private method is a detail of the public method. The clock is an input, either passed in or read from the system.

Situation	How the test reaches it
Public static pure function	Call ClassName.method(arguments). Do not mock it.
Package-private static	Same call, from a test in that package. Prefer an instance when the type has collaborators.
Private method	Call the public method that uses it.
Time passed in, or a Clock parameter	Pass Clock.fixed(...) or a fixed Instant.
Time read with Instant.now() inside	Use instants seconds away from the real now, or change the design to take a Clock.

`FeeCalculator.total` is static because it has no fields and no repository. The test calls `FeeCalculator.total(5, 100)`. Mockito can rewrite bytecode and stub a static method. That tool is for a static call you do not own. This method is ours. A different total is a different pair of arguments.

A private helper is reached through the public method. If `count` hides `normalise`, then `count_null_returnsZero` and `count_extraSpaces_returnsTwo` both run that helper. Reflection goes red when the helper is renamed, even when `count` still returns 0 and 2.

Prefer a `Clock` argument over a hidden `Instant.now()`. The test then owns the instant, and one second is an exact edge.

```
public final class Deadline {
    private final Clock clock;

    public Deadline(Clock clock) {
        this.clock = clock;
    }

    public boolean isOverdue(Instant due) {
        return clock.instant().isAfter(due);
    }
}
```

```
private final Clock clock = Clock.fixed(
        Instant.parse("2026-06-01T00:00:00Z"), ZoneOffset.UTC);

@Test
void isOverdue_dueOneSecondAhead_returnsFalse() {
    // Arrange
    Deadline deadline = new Deadline(clock);
    Instant due = Instant.parse("2026-06-01T00:00:01Z");
    // Act
    boolean overdue = deadline.isOverdue(due);
    // Assert
    assertThat(overdue).isFalse();
}
```

When a method you do not control calls `Instant.now()` itself, choose expiries five seconds in the past and sixty seconds in the future. A one-millisecond gap can flip between Arrange and Act. `Thread.sleep` is the wrong tool for that edge, and it is also the wrong tool for asynchronous work. Section 7.10 uses Awaitility for work that completes later.

6. A service unit test

The service holds the rule. The repository holds rows. Spring Data already implements `save` and `findById`. You do not unit-test that interface. You mock it, call the real service, and check two things: the value returned, and that the repository method ran once. A mocked repository does not prove a mapping, a Flyway script, or a constraint.

```
@ExtendWith(MockitoExtension.class)
@Tag("unit")
class PaymentServiceTest {

    @Mock PaymentRepository payments;
    @Mock Clock clock;
    @InjectMocks PaymentService service;

    @Test
    void charge_positiveAmount_savesThePayment() {
        // Arrange
        Payment saved = new Payment(1L, 500);
        when(payments.save(any(Payment.class))).thenReturn(saved);

        // Act
        Payment result = service.charge(500);

        // Assert
        assertThat(result.id()).isEqualTo(1L);
        assertThat(result.amount()).isEqualTo(500);
        verify(payments, times(1)).save(any(Payment.class));
    }

    @Test
    void charge_zeroAmount_doesNotSave() {
        // Act + Assert
        assertThatThrownBy(() -> service.charge(0))
                .isInstanceOf(IllegalArgumentException.class);
        verifyNoInteractions(payments);
    }
}
```

Piece	Job
@ExtendWith(MockitoExtension.class)	Creates every @Mock before the test. @Mock does nothing without it.
@Mock	A stand-in. Unstubbed methods return a default.
@InjectMocks	Builds PaymentService and passes the mocks into the constructor.
when(...).thenReturn(...)	Arrange. The next matching call returns this value.
verify(..., times(1))	Assert. That method ran once.
verifyNoInteractions(...)	Assert. No method on that mock ran.

Stubbing a calculator to return 5 does not prove addition. `CalculatorTest` proves addition. The service test proves that `charge` saves whatever the collaborator returned, and that a zero amount never reaches `save`. The class under test stays real. A mock of `PaymentService` inside `PaymentServiceTest` checks the mock.

7. Integration tests

An integration test starts the boundary you want confidence in: MVC and validation, JPA and PostgreSQL, or the full application with security. Use the smallest context that can expose the bug.

7.1 Annotations on Spring Boot 4

This stack is Spring Boot 4. The test starters in the build are `spring-boot-starter-webmvc-test`, `spring-boot-starter-data-jpa-test`, `spring-boot-starter-security-test`, and `spring-boot-testcontainers`. They bring JUnit 5, AssertJ, Mockito, MockMvc, and the slice annotations.

`@MockitoBean` has been the way to replace a Spring bean with a mock since Spring Boot 3.4, when it took the place of `@MockBean`. Boot 4 keeps `@MockitoBean`. A tutorial that still shows `@MockBean` does not match this stack. `@Mock` remains the annotation for a unit test that has no Spring context.

Annotation	Package on Boot 4	What starts
@WebMvcTest	org.springframework.boot .webmvc.test.autoconfigure	MVC and one controller. You supply the service.
@AutoConfigureMockMvc	org.springframework.boot .webmvc.test.autoconfigure	MockMvc on an existing @SpringBootTest. No listen port.
@DataJpaTest	org.springframework.boot .data.jpa.test.autoconfigure	JPA, and a rollback around each test.
@AutoConfigureTestDatabase	org.springframework.boot .jdbc.test.autoconfigure	replace = NONE keeps your datasource.
@MockitoBean	org.springframework.test .context.bean.override.mockito	Replaces one bean in a running context.
@ServiceConnection	org.springframework.boot .testcontainers.service.connection	Wires a container as the datasource or Redis.
@SpringBootTest	org.springframework.boot .test.context	The application context.

7.2 Controller slice

The slice proves the mapping. It does not prove the arithmetic, because the mock returns 9 even if `add` is deleted. Keep `CalculatorTest` for the sum.

```
@WebMvcTest(FeeController.class)
@Tag("integration")
class FeeControllerSliceTest {

    @Autowired MockMvc mvc;
    @MockitoBean FeeService fees;

    @Test
    void total_stubbedSum_writesThatSum() throws Exception {
        // Arrange
        when(fees.quote(2, 3)).thenReturn(9L);

        // Act
        var response = mvc.perform(get("/api/fees")
                .param("quantity", "2")
                .param("unitPrice", "3"));

        // Assert
        response.andExpect(status().isOk())
                .andExpect(jsonPath("$.total").value(9));
    }
}
```

7.3 A container is the engine you ship

Testcontainers is a JUnit extension that asks Docker for a real server, waits until that server accepts connections, and hands your test the host and the random port. When the JVM exits, the containers stop. The test talks to PostgreSQL 16 and Redis 7, the same engines the application ships with.

An embedded database is a different product. It accepts different types, different JSON, and different locking, and a Flyway script written for PostgreSQL stops on the first statement it does not understand. A repository unit test with a mock never opens a connection at all. The container is the first level that can go red for a bad column type, a unique index, or a query the mock would have agreed with.

```
JUnit starts the test class
        |
        v
Docker starts postgres:16 and redis:7
on a random host port each
        |
        v
@ServiceConnection registers the URL,
the user, the password, the Redis host, the Redis port
        |
        v
Spring starts. Flyway migrates when the
full application is what you started.
        |
        v
The test methods run
        |
        v
The JVM exits and the containers stop
```

Setup	Engine	What a failure means
@DataJpaTest alone	Embedded, if one is on the classpath	The mapping shape. The SQL may still be wrong on PostgreSQL.
@DataJpaTest, replace = NONE, plus a PostgreSQL container	PostgreSQL 16	The query, the column types, or a constraint.
@SpringBootTest plus the containers	PostgreSQL 16 and Redis 7	Flyway, security, a commit, or a key that must expire.

The first run pulls the images and is slow. Later runs reuse the images already on the machine. The host port changes every run. The test reads that port from the container. It leaves the database on port 5432, the one from Compose, alone.

7.4 The annotations, and the block they replace

`@Testcontainers` registers the JUnit extension. `@Container` marks the field the extension starts. A `static` field starts once for the class and stays up for every method. An instance field starts and stops around every method, which rebuilds the database for each test and throws away the schema you just migrated.

`@ServiceConnection` reads the running container and registers the connection. The five lines below are what that annotation writes for you. A JWT secret, a schema name, or a feature flag is still yours to set, in `@TestPropertySource`, because those are not a service connection.

```
@DynamicPropertySource
static void wire(DynamicPropertyRegistry registry) {
    registry.add("spring.datasource.url", POSTGRES::getJdbcUrl);
    registry.add("spring.datasource.username", POSTGRES::getUsername);
    registry.add("spring.datasource.password", POSTGRES::getPassword);
    registry.add("spring.data.redis.host", REDIS::getHost);
    registry.add("spring.data.redis.port",
            () -> REDIS.getMappedPort(6379));
}
```

The Redis image name has to be `redis`. Boot 4's Redis connection factory recognises `redis`, `redis/redis-stack`, and `redis/redis-stack-server`. An image under another name starts, and Spring never points the client at it. `postgres:16-alpine` is recognised as PostgreSQL.

`disabledWithoutDocker = true` skips the class when the daemon is down, and `./mvnw test` can still report success. Run the integration group with Docker up and read the test count. A skipped container is a missing assertion.

7.5 A repository slice against PostgreSQL

This is the repository row of the diagram. One repository, real PostgreSQL, no web server, no Redis. `@DataJpaTest` replaces your datasource with an embedded database unless you stop it. `replace = NONE` keeps the container. `ddl-auto=create-drop` builds the tables from the entities for this slice.

The slice does not run Flyway. A failure that says the table does not exist means the connection is real and the schema was never created. Section 7.7 starts the full application, and that is where the migration scripts run.

`StockItem` maps `sku` as the primary key and `quantity` as an int. `save` can hold the insert until the transaction ends. `saveAndFlush` sends it now, which is what makes a unique-key failure visible inside the test. The slice then rolls the transaction back, so the next method starts from an empty table.

```
@DataJpaTest
@AutoConfigureTestDatabase(replace = NONE)
@TestPropertySource(properties =
        "spring.jpa.hibernate.ddl-auto=create-drop")
@Testcontainers(disabledWithoutDocker = true)
@Tag("integration")
class InventoryRepositoryTest {

    @Container
    @ServiceConnection
    static final PostgreSQLContainer<?> POSTGRES =
            new PostgreSQLContainer<>(
                    DockerImageName.parse("postgres:16-alpine"));

    @Autowired InventoryRepository inventory;

    @Test
    void save_newSku_canBeLoaded() {
        // Arrange
        inventory.saveAndFlush(new StockItem("USB-C-01", 12));

        // Act
        StockItem found = inventory.findBySku("USB-C-01")
                .orElseThrow();

        // Assert
        assertThat(found.quantity()).isEqualTo(12);
        assertThat(POSTGRES.getJdbcUrl())
                .startsWith("jdbc:postgresql:");
    }

    @Test
    void save_duplicateSku_isRejected() {
        // Arrange
        inventory.saveAndFlush(new StockItem("USB-C-01", 12));

        // Act + Assert
        assertThatThrownBy(() -> inventory.saveAndFlush(
                new StockItem("USB-C-01", 1)))
                .isInstanceOf(DataIntegrityViolationException.class);
    }
}
```

On Boot 4 the two Spring imports below moved. A snippet copied from an older tutorial still uses the Boot 3 packages and does not compile until these replace them.

```
import org.springframework.boot.data.jpa.test
        .autoconfigure.DataJpaTest;
import org.springframework.boot.jdbc.test
        .autoconfigure.AutoConfigureTestDatabase;
import org.springframework.boot.testcontainers
        .service.connection.ServiceConnection;
import org.testcontainers.containers.PostgreSQLContainer;
import org.testcontainers.junit.jupiter.Container;
import org.testcontainers.junit.jupiter.Testcontainers;
import org.testcontainers.utility.DockerImageName;
```

7.6 Redis, the same idea

A blacklist that only lives in a `HashMap` forgets its entries on restart and has no expiry. The test that proves the key and the TTL uses the Redis the application uses. This one needs the full context, because a JPA slice does not start the Redis client. It extends the base class from the next section, so it does not declare a second container.

```
@SpringBootTest
@Tag("integration")
class TokenBlacklistRedisTest extends AbstractIntegrationTest {

    @Autowired StringRedisTemplate redis;

    @AfterEach
    void removeTheKeyThisTestWrote() {
        redis.delete("blacklist:abc");
    }

    @Test
    void revoke_liveToken_isStoredWithATtl() {
        // Act
        redis.opsForValue().set(
                "blacklist:abc", "1", Duration.ofMinutes(30));

        // Assert
        assertThat(redis.hasKey("blacklist:abc")).isTrue();
        Long seconds = redis.getExpire(
                "blacklist:abc", TimeUnit.SECONDS);
        assertThat(seconds).isPositive();
    }

    @Test
    void lookup_unknownToken_isAbsent() {
        // Assert
        assertThat(redis.hasKey("blacklist:missing")).isFalse();
    }
}
```

`hasKey` true with a positive TTL means the command reached Redis. A mock of the template would have returned whatever the stub said, including a TTL for a key that was never stored. The absent-key test is the other side: a missing token is not blacklisted.

7.7 One pair of containers for the suite

Declare PostgreSQL and Redis once, as static fields on a base class. The field lives for the whole JVM, and `start()` on a container that is already running does not pull a second database. Each concrete class still builds its own Spring context, pointed at that same pair. The base carries `@Testcontainers`. `@ExtendWith` on that annotation is inherited, so a subclass does not repeat it.

```
@Testcontainers(disabledWithoutDocker = true)
public abstract class AbstractIntegrationTest {

    @Container
    @ServiceConnection
    static final PostgreSQLContainer<?> POSTGRES =
            new PostgreSQLContainer<>(
                    DockerImageName.parse("postgres:16-alpine"));

    @Container
    @ServiceConnection
    static final GenericContainer<?> REDIS =
            new GenericContainer<>(
                    DockerImageName.parse("redis:7-alpine"))
                    .withExposedPorts(6379);
}
```

The full context is where Flyway runs. This test posts an order and then reads the row back. The HTTP status proves the controller. The repository read proves PostgreSQL kept the row. A second post with the same idempotency key must return the same order and leave a single row. That rule belongs here. The browser test in section 9 checks that a person can finish, and that the screen matches this row.

```
@SpringBootTest
@AutoConfigureMockMvc
@Tag("integration")
class PaymentFlowTest extends AbstractIntegrationTest {

    @Autowired MockMvc mvc;
    @Autowired OrderRepository orders;

    @AfterEach
    void removeRowsThisTestCommitted() {
        orders.deleteAll();
    }

    @Test
    void checkout_oneItem_persistsPlacedOrder() throws Exception {
        // Arrange
        String body = "{\"sku\":\"USB-C-01\",\"quantity\":1}";

        // Act
        mvc.perform(post("/api/orders")
                .contentType(MediaType.APPLICATION_JSON)
                .header("Idempotency-Key", "order-usb-1")
                .content(body))
                .andExpect(status().isCreated())
                .andExpect(jsonPath("$.status").value("PLACED"));

        // Assert: the row is in PostgreSQL
        assertThat(orders.findAll()).singleElement()
                .extracting(Order::sku, Order::status)
                .containsExactly("USB-C-01", "PLACED");
    }

    @Test
    void checkout_sameKey_returnsTheOriginalOrder() throws Exception {
        // Arrange
        String body = "{\"sku\":\"USB-C-01\",\"quantity\":1}";
        String key = "order-usb-1";
        String firstId = idOf(postOrder(body, key));

        // Act
        String secondId = idOf(postOrder(body, key));

        // Assert
        assertThat(secondId).isEqualTo(firstId);
        assertThat(orders.findAll()).hasSize(1);
    }
}
```

`postOrder` performs the same POST and returns the response body. `idOf` reads `$.id`. Two methods keep the replay test about one fact: the same key is the same order. `@AfterEach` deletes the rows this class committed. Section 7.9 is the full set of cleanliness habits, including why a `@SpringBootTest` does not roll back on its own.

7.8 Reading a container failure

What the run prints	What it means
Could not find a valid Docker environment	The daemon is stopped, or DOCKER_HOST points at a socket the process cannot open.
Container startup failed for postgres:16-alpine	The pull failed, or the daemon has no disk left. The test never reached Spring.
Connection to localhost:5432 refused	The URL was typed by hand. @ServiceConnection is missing, or the Redis image name was not redis, so Boot ignored that container.
relation "order" does not exist	PostgreSQL is up and the migration did not run. A slice with create-drop builds tables from entities. The full context is what runs Flyway.
Tests run: 0, skipped	disabledWithoutDocker skipped the class. The build is green and the assertion never ran.

7.9 Keeping rows from leaking into the next test

`@DataJpaTest` wraps each test in a transaction and rolls it back. `@SpringBootTest` does not. A row committed by one method is still there for the next method in that class. That is how successive tests pollute each other.

Three habits keep a full-context test isolated. Use the first by default. Use the second when the test must observe a real commit. Use the third when the schema itself is what you reset.

- **Unique keys.** Each test creates its own customer and its own stock item. It does not edit a seeded row that other tests still need.
- **Delete what you wrote.** An `@AfterEach` method deletes child rows before parent rows: payments, then orders, then customers. `deleteAll` on the parent first fails on the foreign key.
- **Rebuild the schema.** `flyway.clean()` then `flyway.migrate()` drops and recreates the tables. Set `spring.flyway.clean-disabled=false` in the test properties, because current Flyway refuses a clean until you allow it. An `ApplicationRunner` that seeded data at startup does not run again after `clean`, because the Spring context is still up. Put seed data in a Flyway migration, or call the seeder again yourself.

```
@AfterEach
void removeRowsThisTestCommitted() {
    payments.deleteAll();
    orders.deleteAll();
    customers.deleteAll();
}
```

You can put `@Transactional` on a `@SpringBootTest` method. Spring's test listener then rolls back the transaction on the test thread, and `MockMvc` joins that thread, so the rollback covers those requests. Leave that annotation off when the behaviour under test is "this write survives the exception." A rollback would hide the bug. `RANDOM_PORT` is a different thread: the server commits in its own transaction, and a `@Transactional` test does not roll that commit back. Read the row after the call.

A login that increments `failedAttempts` and then throws is the case the rollback hides. If the service is `@Transactional` and the exception rolls the increment back, the account never locks. `noRollbackFor` on that exception keeps the save. The test loads the user again after the third failure and asserts the locked status. The instance you held before the request does not show the commit.

7.10 Asynchronous work

A timestamp edge uses a fixed clock, as section 5.3 shows. A scheduled job, a message listener, or any hand-off to another thread completes later. Waiting with `Thread.sleep` either wastes time or finishes before the work does.

Add Awaitility in test scope. The Spring Boot parent manages the version.

```
<dependency>
    <groupId>org.awaitility</groupId>
    <artifactId>awaitility</artifactId>
    <scope>test</scope>
</dependency>
```

```
@Test
void restock_afterPublish_reachesZeroReserved() {
    // Arrange
    inventory.reserve(sku, 2);
    publisher.restock(sku);

    // Assert: poll until the listener has applied the message
    await().atMost(Duration.ofSeconds(2))
            .untilAsserted(() ->
                    assertThat(inventory.reserved(sku)).isZero());
}
```

`await().atMost(...).untilAsserted(...)` polls. It fails with the assertion message when the deadline passes. It does not replace a unit test of the restock rule. It waits for the boundary between the publisher and the listener.

8. JSON over HTTP

A controller test that compares the whole response to one string breaks when a field moves or a space changes. JSONPath names the field. The assertion stays on the contract: this id, this name, this installment, and no dependence on key order.

The sample endpoint is `GET /api/loans/101`. A known loan returns this document. The tests below assert pieces of it. None of them pass because a developer database already happens to contain loan 101. `@BeforeEach` saves that loan into the test database, or the slice stubs the service so the database is not involved.

```
{
  "id": 101,
  "customerName": "Jean Paul",
  "principal": 1000000,
  "status": "ACTIVE",
  "eligible": true,
  "installments": [
    { "number": 1, "amount": 85000 },
    { "number": 2, "amount": 85000 }
  ]
}
```

8.1 The paths

`$` is the root. A dot steps into a field. Brackets pick an array element, starting at zero. `length()` is the array size in Jayway JSONPath, which is what `jsonPath()` on MockMvc uses. Boot manages that library. `$.installments[-1]` is the last element.

What you want	JSONPath	Value in the sample
Root id	$.id	101
A string field	$.customerName	Jean Paul
A boolean	$.eligible	true
First installment	$.installments[0].amount	85000
Last installment	$.installments[-1].number	2
How many	$.installments.length()	2

REST Assured's `.body()` checks use Groovy GPath, not JSONPath. The ideas match. The spelling does not. There is no `$`, and the size of a list is `size()`.

Fact	MockMvc jsonPath	REST Assured body
Id	$.id	id
First amount	$.installments[0].amount	installments[0].amount
How many	$.installments.length()	installments.size()

8.2 Three reaches, three levels

The same JSON can be asserted from three places. The level is still the boundary, not the matcher. `jsonPath()` does not make a test end to end, and a browser is not required to call the API.

Reach	What actually runs	Level
@WebMvcTest + jsonPath	One controller. The service is a @MockitoBean. No port, no database.	Integration, the web slice
@SpringBootTest + MockMvc + jsonPath	The full context and the containers. The call never leaves the process.	Integration
REST Assured + RANDOM_PORT	A real socket, the filters, the controller, the service, the repository, PostgreSQL.	End to end, HTTP

Keep most field checks on the slice, where they fail in milliseconds. Use the full context when the row has to exist. Send a few critical reads through the real port, where a filter or a message converter can still be wrong. Section 9 opens the page. A green JSON test can sit beside a red button.

8.3 The slice

The controller below returns whatever `LoanService.find` returns. The slice starts that controller and a mock of the service. `activeLoan()` is the document from the top of this section, built in Java. Deleting the repository leaves this test green, which is the point: the mapping and the JSON names are under test, and the arithmetic of eligibility is not.

```
@RestController
@RequestMapping("/api/loans")
class LoanController {
    private final LoanService loans;

    public LoanController(LoanService loans) {
        this.loans = loans;
    }

    @GetMapping("/{id}")
    Loan get(@PathVariable long id) {
        return loans.find(id);
    }
}
```

```
@WebMvcTest(LoanController.class)
@Tag("integration")
class LoanControllerSliceTest {

    @Autowired MockMvc mvc;
    @MockitoBean LoanService loans;

    @Test
    void get_knownId_returnsTheContract() throws Exception {
        // Arrange
        when(loans.find(101L)).thenReturn(activeLoan());

        // Act + Assert
        mvc.perform(get("/api/loans/101")
                .accept(MediaType.APPLICATION_JSON))
            .andExpect(status().isOk())
            .andExpect(content().contentTypeCompatibleWith(
                    MediaType.APPLICATION_JSON))
            .andExpect(jsonPath("$.id").value(101))
            .andExpect(jsonPath("$.customerName")
                    .value("Jean Paul"))
            .andExpect(jsonPath("$.principal").value(1000000))
            .andExpect(jsonPath("$.status").value("ACTIVE"))
            .andExpect(jsonPath("$.eligible").value(true))
            .andExpect(jsonPath("$.installments.length()")
                    .value(2))
            .andExpect(jsonPath("$.installments[0].number")
                    .value(1))
            .andExpect(jsonPath("$.installments[0].amount")
                    .value(85000))
            .andExpect(jsonPath("$.installments[-1].number")
                    .value(2));
    }
}
```

If this call comes back 401, the security filter is inside the slice and the request has no token. That 401 is a separate test: a request with no token is unauthorized. Do not turn the filters off to make the JSON assertion pass. Send the header the API requires, and keep the JSON checks.

`@WebMvcTest(LoanController.class)` does not load a `@ControllerAdvice` from another class unless you `@Import` it. An error body asserted on the slice is only as real as the advice you imported. The next section has the advice, because it starts the application.

8.4 The full context, still inside the process

This test extends `AbstractIntegrationTest`, so PostgreSQL is the container from section 7.7 and Flyway has run. `MockMvc` dispatches the request on the test thread. Nothing listens on a port. The JSON matchers are the same ones as the slice. The new fact is that loan 101 was saved, and a missing id becomes the error document your handler writes.

```
@SpringBootTest
@AutoConfigureMockMvc
@Tag("integration")
class LoanDetailsTest extends AbstractIntegrationTest {

    @Autowired MockMvc mvc;
    @Autowired LoanRepository loans;

    @BeforeEach
    void arrangeLoan101() {
        loans.deleteAll();
        Loan loan = new Loan(
                101L, "Jean Paul", 1_000_000, "ACTIVE", true);
        loan.addInstallment(1, 85_000);
        loan.addInstallment(2, 85_000);
        loans.save(loan);
    }

    @AfterEach
    void removeRowsThisTestCommitted() {
        loans.deleteAll();
    }

    @Test
    void get_knownId_returnsTheContract() throws Exception {
        // Act + Assert
        mvc.perform(get("/api/loans/101"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.customerName")
                    .value("Jean Paul"))
            .andExpect(jsonPath("$.installments.length()")
                    .value(2))
            .andExpect(jsonPath("$.installments[0].amount")
                    .value(85000));
    }
}
```

A missing loan is part of the contract. Assert the status and the fields the handler writes. If your API uses `detail` instead of `message`, assert `detail`. The path has to name the field you ship.

```
@Test
void get_missingId_returnsNotFound() throws Exception {
    // Act + Assert
    mvc.perform(get("/api/loans/999999"))
        .andExpect(status().isNotFound())
        .andExpect(jsonPath("$.status").value(404))
        .andExpect(jsonPath("$.message")
                .value("Loan not found"));
}
```

8.5 A null field is not a missing field

Spring's `jsonPath` matchers on this stack distinguish four situations. `exists()` requires a non-null value, and an empty array still counts. `doesNotExist()` is the opposite: the value is null or the path is absent. `hasJsonPath()` accepts a present null. `doesNotHaveJsonPath()` requires the key to be absent, so a stored null fails it. `isEmpty()` and `isNotEmpty()` follow `ObjectUtils.isEmpty`: a null, `""`, an empty array, and an empty object are empty. A boolean and a number are not.

```
// "customerName": "Jean Paul"
jsonPath("$.customerName").exists();
jsonPath("$.customerName").isNotEmpty();

// "middleName": null   the key is present
jsonPath("$.middleName").hasJsonPath();
jsonPath("$.middleName").doesNotExist();

// the key was omitted
jsonPath("$.middleName").doesNotHaveJsonPath();

// "nickname": ""
jsonPath("$.nickname").exists();
jsonPath("$.nickname").isEmpty();
```

`doesNotExist()` and `isEmpty()` are different promises. Pick the one the API owes the client. A field that must be omitted, a field that must be null, and a field that must be an empty list are three tests.

8.6 REST Assured on a real port

This is the HTTP end-to-end test. `@SpringBootTest(webEnvironment = RANDOM_PORT)` binds a free port. REST Assured sends a real request to it. The call passes the socket, the security filters, the controller, the service, the repository, and PostgreSQL from `AbstractIntegrationTest`. `MockMvc` never does that.

Spring Boot 4 does not manage REST Assured, so the version stays in the pom. 5.5.7 is the line this sample uses. The test annotation `@LocalServerPort` lives in `org.springframework.boot.test.web.server`.

```
<dependency>
    <groupId>io.rest-assured</groupId>
    <artifactId>rest-assured</artifactId>
    <version>5.5.7</version>
    <scope>test</scope>
</dependency>
```

```
@SpringBootTest(webEnvironment = RANDOM_PORT)
@Tag("e2e")
class LoanHttpTest extends AbstractIntegrationTest {

    @LocalServerPort int port;
    @Autowired LoanRepository loans;

    @BeforeEach
    void arrangeLoan101() {
        loans.deleteAll();
        Loan loan = new Loan(
                101L, "Jean Paul", 1_000_000, "ACTIVE", true);
        loan.addInstallment(1, 85_000);
        loan.addInstallment(2, 85_000);
        loans.save(loan);
    }

    @AfterEach
    void removeRowsThisTestCommitted() {
        loans.deleteAll();
    }

    @Test
    void get_knownId_returnsTheContract() {
        // Act + Assert. Paths are GPath, not JSONPath.
        given()
            .port(port)
            .accept(ContentType.JSON)
        .when()
            .get("/api/loans/101")
        .then()
            .statusCode(200)
            .contentType(ContentType.JSON)
            .body("id", equalTo(101))
            .body("customerName", equalTo("Jean Paul"))
            .body("principal", equalTo(1000000))
            .body("status", equalTo("ACTIVE"))
            .body("eligible", equalTo(true))
            .body("installments.size()", equalTo(2))
            .body("installments[0].number", equalTo(1))
            .body("installments[0].amount", equalTo(85000))
            .body("installments[1].number", equalTo(2))
            .body("installments[1].amount", equalTo(85000));
    }

    @Test
    void get_missingId_returnsNotFound() {
        given()
            .port(port)
        .when()
            .get("/api/loans/999999")
        .then()
            .statusCode(404)
            .body("status", equalTo(404))
            .body("message", equalTo("Loan not found"));
    }
}
```

`equalTo` and `ContentType` come from Hamcrest and REST Assured: `io.restassured.RestAssured.given`, `io.restassured.http.ContentType`, `org.hamcrest.Matchers.equalTo`. AssertJ stays on values you hold in Java, such as a row read back from the repository. A secured endpoint adds `.header("Authorization", "Bearer " + token)` on the `given()`. A 401 means the filter ran, and the JSON paths never got a body to read.

Point this client at the port the test opened. A URL of `localhost:8080`, or a database from a developer's Compose file, makes the test depend on whatever was left there.

8.7 What Playwright still has to do

REST Assured and Playwright can share `RANDOM_PORT` and the same containers. They do not share a job. One reads JSON. The other uses the page.

	REST Assured	Playwright
Drives	The API URL	The page a person sees
Asserts	Status, content type, JSON fields	Labels, buttons, the URL, and the row
Goes red when	A field, a filter, or a converter is wrong	The button posts to the wrong path, or the screen disagrees with the row
Stays green when	The page has no Pay button	A JSON field the screen never shows is renamed

Loan 101's contract belongs in this section. The journey that signs in and sees ACTIVE on the screen belongs in section 9. Tag the HTTP class `e2e` with the browser class: both open a real port. The slice and the MockMvc class stay `integration`.

9. End-to-end tests with Playwright

Section 8 already proved the JSON fields. An end-to-end test of the page checks one journey a person can complete, through the stack that person touches: the page, the gateway or dev proxy, the Spring Boot API, and the database or Redis behind it. It is the test that goes red when the button posts to the wrong path, when the proxy drops a header, or when the screen shows a status the database does not hold.

The ten-percent fee already has a unit test. The idempotency key already has the MockMvc test in section 7.7. The browser test checks that a person can finish the journey and that the page shows the persisted outcome. If the page sends a new key on every click, this journey goes red because a second order appears.

Picture the bug this level is for. `FeeCalculatorTest` passes. `PaymentFlowTest` passes. The screen stays on PLACED. The button posted to `/api/order`, and the mapping is `/api/orders`. Playwright is the test that clicks that button.

9.1 The dependency and the browser

Playwright for Java is `com.microsoft.playwright:playwright`, test scope, on the current 1.x line. The jar does not contain Chromium. On a laptop, the first `chromium().launch()` downloads it. In the pipeline the image has no browser, so the job installs Chromium before the tests, as section 11 shows.

```
<dependency>
    <groupId>com.microsoft.playwright</groupId>
    <artifactId>playwright</artifactId>
    <version>1.63.0</version>
    <scope>test</scope>
</dependency>
```

Pin the version you have verified. 1.63.0 is a concrete 1.x coordinate so the snippet resolves. Move it forward when you upgrade the suite.

9.2 How a locator finds the element

The page under test looks like this. The locators use the label, the button's accessible name, and a test id the team owns. A CSS class changes when the layout changes, and the journey should stay green.

```
<label>
  Email
  <input name="email" />
</label>
<label>
  Password
  <input type="password" name="password" />
</label>
<button type="submit">Sign in</button>
<p data-testid="order-status">PAID</p>
```

You want	Call	It matches
The email field	getByLabel("Email")	The input inside that label.
The sign-in button	getByRole(BUTTON, name "Sign in")	The accessible name, in the language of the page.
The status the page owns	getByTestId("order-status")	The data-testid attribute.
The address after a redirect	PlaywrightAssertions.assertThat(page).hasURL(...)	The URL, including the random port.

Playwright waits. `click()` waits until the button is visible and enabled. A page assertion retries until the text appears or the timeout fires. AssertJ reads a value once. The two libraries both offer `assertThat`, and a static import of both does not compile. Keep AssertJ as the static import. Call the page assertions by their class.

```
import static org.assertj.core.api.Assertions.assertThat;

// Retries until the text is PAID, or the timeout fires.
PlaywrightAssertions.assertThat(
        page.getByTestId("order-status")).hasText("PAID");

// Reads PostgreSQL once. The request has already committed.
assertThat(payments.findAll()).hasSize(1);
```

Set a short default in `@BeforeEach` so a missing button fails in five seconds during class. A timeout means the locator never matched: the route is wrong, the label text changed, or the API never answered. A `Thread.sleep` in front of the same locator waits the wrong length and still fails when the name is wrong. `strict mode violation` means two elements share that name. Tighten the name, or scope the locator to the form.

9.3 One class, the port, and the browser

`@SpringBootTest(webEnvironment = RANDOM_PORT)` starts the API on a free port. `@LocalServerPort` injects that port after the context is up. A static `@BeforeAll` runs too early to see it. `@TestInstance(PER_CLASS)` lets `@BeforeAll` be an instance method, so the browser starts once, with the port already known. The class extends `AbstractIntegrationTest`, so PostgreSQL and Redis are the containers from section 7.7.

In this sample Boot serves the page. The browser opens `http://127.0.0.1:` plus `apiPort`. Section 9.5 starts a separate frontend against the same port.

```
@SpringBootTest(webEnvironment = RANDOM_PORT)
@TestInstance(Lifecycle.PER_CLASS)
@Tag("e2e")
class CheckoutJourneyTest extends AbstractIntegrationTest {

    @LocalServerPort int apiPort;
    @Autowired PaymentRepository payments;

    Playwright playwright;
    Browser browser;
    BrowserContext context;
    Page page;

    @BeforeAll
    void launch() {
        playwright = Playwright.create();
        browser = playwright.chromium().launch(
                new BrowserType.LaunchOptions().setHeadless(true));
    }

    @BeforeEach
    void openTab() {
        context = browser.newContext();
        page = context.newPage();
        page.setDefaultTimeout(5_000);
    }

    @AfterEach
    void closeTab() {
        context.close();
        payments.deleteAll();
    }

    @AfterAll
    void shutdown() {
        browser.close();
        playwright.close();
    }

    String url(String path) {
        return "http://127.0.0.1:" + apiPort + path;
    }
}
```

9.4 Four journeys

Each method is one outcome. The happy path signs in, opens a product, pays, and checks the screen against the row. The other three go red for a different bug: a missing redirect, a validation message the API returned and the page swallowed, and a declined stub that the screen calls paid.

```
@Test
void checkout_validCustomer_showsPaidOrder() {
    // Arrange
    page.navigate(url("/login"));
    page.getByLabel("Email").fill("ada@shop.test");
    page.getByLabel("Password").fill("Secret1a");

    // Act
    page.getByRole(AriaRole.BUTTON,
            new Page.GetByRoleOptions().setName("Sign in")).click();
    page.getByRole(AriaRole.LINK,
            new Page.GetByRoleOptions()
                    .setName("USB-C cable")).click();
    page.getByLabel("Quantity").fill("1");
    page.getByLabel("Payment result").selectOption("SUCCESS");
    page.getByRole(AriaRole.BUTTON,
            new Page.GetByRoleOptions().setName("Pay")).click();

    // Assert: the screen and the row agree
    PlaywrightAssertions.assertThat(
            page.getByTestId("order-status")).hasText("PAID");
    assertThat(payments.findAll()).singleElement()
            .extracting(Payment::status)
            .isEqualTo("PAID");
}
```

```
@Test
void checkout_anonymous_landsOnLogin() {
    // Act
    page.navigate(url("/checkout"));

    // Assert
    PlaywrightAssertions.assertThat(page).hasURL(
            Pattern.compile(".*/login.*"));
    assertThat(payments.findAll()).isEmpty();
}

@Test
void signIn_blankEmail_showsTheMessage() {
    // Arrange
    page.navigate(url("/login"));

    // Act
    page.getByRole(AriaRole.BUTTON,
            new Page.GetByRoleOptions().setName("Sign in")).click();

    // Assert
    PlaywrightAssertions.assertThat(
            page.getByText("Email is required")).isVisible();
}
```

```
@Test
void checkout_declinedStub_staysUnpaid() {
    // Arrange
    page.navigate(url("/login"));
    page.getByLabel("Email").fill("ada@shop.test");
    page.getByLabel("Password").fill("Secret1a");
    page.getByRole(AriaRole.BUTTON,
            new Page.GetByRoleOptions().setName("Sign in")).click();
    page.getByLabel("Payment result").selectOption("DECLINE");

    // Act
    page.getByRole(AriaRole.BUTTON,
            new Page.GetByRoleOptions().setName("Pay")).click();

    // Assert
    PlaywrightAssertions.assertThat(
            page.getByTestId("order-status")).hasText("DECLINED");
    assertThat(payments.findByStatus("PAID")).isEmpty();
}
```

On a failure, write a screenshot next to the assertion message. Open it when the text is wrong and you need to see what the page actually rendered.

```
page.screenshot(new Page.ScreenshotOptions()
        .setPath(Path.of("target/checkout.png"))
        .setFullPage(true));
```

9.5 A separate frontend in the same job

When the page is a Vite or similar dev server, the test still owns the API port. `@BeforeAll` starts that server with the proxy aimed at `apiPort`, waits until port 5173 accepts a connection, and only then opens a tab. `@AfterAll` destroys the process. The journey methods navigate to `http://127.0.0.1:5173` and keep the same locators. The pipeline does not depend on a server someone left on 8080.

```
Process ui;

@BeforeAll
void launch() throws Exception {
    playwright = Playwright.create();
    browser = playwright.chromium().launch(
            new BrowserType.LaunchOptions().setHeadless(true));
    ProcessBuilder vite = new ProcessBuilder(
            "npm", "run", "dev", "--",
            "--port", "5173", "--strictPort");
    vite.directory(new File("frontend"));
    vite.environment().put(
            "API_PROXY", "http://127.0.0.1:" + apiPort);
    vite.redirectErrorStream(true);
    ui = vite.start();
    await().atMost(Duration.ofSeconds(20)).until(this::uiUp);
}

boolean uiUp() {
    try (Socket socket = new Socket("127.0.0.1", 5173)) {
        return true;
    } catch (IOException ex) {
        return false;
    }
}

@AfterAll
void shutdown() {
    ui.destroy();
    browser.close();
    playwright.close();
}
```

The dev server has to read `API_PROXY` and forward `/api` there. A frontend that still proxies to `localhost:8080` makes this job fail, which is the bug the variable exists to catch. Tag the class `e2e`. Once the class exists, drop `failIfNoSpecifiedTests=false` so an empty group cannot pass in silence.

9.6 What belongs in the journey

- A valid customer signs in, pays with the success stub, and sees PAID. One payment row exists, and its status is PAID.
- An anonymous visitor who opens a protected page lands on the login page. The payment table stays empty.
- A blank email shows the validation message the page is supposed to render.
- A declined stub leaves the screen on DECLINED and writes no PAID row.
- The fee table, the idempotency rule, and the repository constraints stay in the lower levels.

10. What a test cannot prove

You cannot prove	Why	What covers the neighbour
A private method, by reflection	The test locks a name. Renaming breaks it while behaviour holds.	The public method that calls it.
SQL, with a mocked repository	The mock never opens PostgreSQL.	A @DataJpaTest or a @SpringBootTest against the container.
Arithmetic, with a controller mock that returns 9	The mock is the result.	FeeCalculatorTest.
A route guard or a dev proxy, with MockMvc	MockMvc never loads the frontend.	The Playwright journey.
That Docker was up, when the class is skipped	A skip reports success.	Run the integration group and read the count.
A real card charge	The gateway in tests is a stub with scripted results.	The stub's success, decline, and timeout.
Pixel layout	No assertion looks at a screenshot unless you add one.	A person, or a visual test you choose to own.
Coverage	JaCoCo counts executed lines.	The assert or verify in the test.

11. Running one level

JUnit `@Tag` marks the class. Surefire reads `-Dgroups`. A tag on the class applies to every method. Surefire runs classes named `*Test`, `*Tests`, and `*TestCase`. A class named `*IT` is compiled and skipped unless you configure Failsafe. This guide uses tags, not the `*IT` suffix.

```
./mvnw -Dgroups=unit test
./mvnw -Dgroups=integration test
./mvnw -Dgroups=e2e test
./mvnw verify
```

`test` runs the selected group. `verify` runs every tag and fails the build when line coverage is under 85 percent. A unit-only run is not supposed to clear that gate, because the controllers never executed. Docker must be running for integration, end-to-end, and `verify`.

A pipeline variable `TEST_LEVEL` takes `all`, `unit`, `integration`, or `e2e`. The unit job has no Docker service. The integration job, the end-to-end job, and the gate job start a Docker daemon beside Maven. Inside that Maven container, `localhost` is not the daemon, so set `TESTCONTAINERS_HOST_OVERRIDE` to the daemon's hostname and `DOCKER_HOST` to its socket. Leave both unset on a laptop. The gate job runs `./mvnw verify` in one process so JaCoCo's single execution file contains every tag.

The end-to-end job is one step. It starts the API on a random port, the containers, and Chromium. It does not call a server someone started by hand.

```
e2e:
  image: maven:3.9-eclipse-temurin-21
  services: [docker:27-dind]
  variables:
    DOCKER_HOST: tcp://docker:2375
    DOCKER_TLS_CERTDIR: ""
    TESTCONTAINERS_HOST_OVERRIDE: docker
  script:
    - >
      ./mvnw -q exec:java
      -Dexec.mainClass=com.microsoft.playwright.CLI
      -Dexec.args="install chromium"
    - ./mvnw -Dgroups=e2e test
  rules:
    - if: $TEST_LEVEL == "e2e" || $TEST_LEVEL == "all"
```

12. Coverage

JaCoCo's check goal belongs on `verify`. Gate line coverage at 0.85 for the bundle. Report branch coverage without making it a gate, until the branch number is something the team will act on. Use the report to find a rule with no test. Do not add a test whose only purpose is to execute a line.

13. Before you call a test finished

- The name is subject_condition_outcome.
- A JSON field is checked with jsonPath or a REST Assured path. The whole document is not compared as one string.
- Arrange, act, and assert are in that order. Asserts use AssertJ.
- A boundary has the success, the line, and the refusal as separate tests.
- The check is a value, a message, a status, or a verify.
- The tag matches the boundary. A test that starts Docker is integration or end to end.
- The test does not need another test to have run first, and @AfterEach removes rows it committed.
- A service takes its collaborators and its Clock in the constructor.
- ./mvnw -Dgroups=unit test is green before you spend time on containers. After the change, ./mvnw verify is green.

14. Questions

One option is correct. Where a question states a fee or shipping rule, answer from that rule.

1. A unit test of Calculator.add starts which of these?

A. The Spring context
B. PostgreSQL
C. JUnit, which calls the method
D. A browser

2. @SpringBootTest plus MockMvc plus Testcontainers is which level?

A. End to end, because the context is complete
B. Integration

C. Unit, because one HTTP call is one method
D. A browser test

3. A Playwright test that signs in, pays, and reads the paid status is which level?

A. Unit
B. Integration, because Spring started
C. End to end
D. A coverage report

4. A visitor who opens a protected page and lands on the login page is proved by which test?

A. MockMvc against the API
B. A browser journey
C. FeeCalculatorTest
D. The JaCoCo report

5. In assertThat(sum).isEqualTo(5), what is sum?

A. The expected result
B. The actual result
C. The tag
D. The failure message

6. Which name matches subject_condition_outcome for "5 divided by 2 returns 2"?

A. testDivide
B. divide_doesNotRoundUp
C. divide_fiveByTwo_returnsTwo
D. returnsTwo

7. A method named charge_positive_savesAndZero_throws should be

A. Left as one method
B. Split into two methods
C. Tagged e2e only
D. Made private

8. count_blankOrNull_returnsZero may stay one method because

A. A class may contain only one @Test
B. Both inputs share one result, and the name says so
C. null cannot be asserted
D. Surefire runs only the first assert

9. Surefire's treatment of a class named CalculatorIT, with no Failsafe plugin, is

A. Run it as an integration test automatically
B. Compile it and skip it
C. Run it when -Dgroups=e2e
D. Run it as a unit test

10. Which command runs only the unit group?

A. ./mvnw verify
B. ./mvnw -Dgroups=unit test
C. ./mvnw -Dgroups=integration test
D. ./mvnw -Dtest=CalculatorIT test

11. ./mvnw -Dgroups=unit test and the 85 percent line gate

A. The gate runs and must pass
B. The gate is bound to verify, so this command does not apply it
C. The gate measures branches only
D. The gate skips every unit class

12. @Tag("unit") on the class means

A. Each method must repeat the tag
B. The methods carry the class tag
C. Spring starts
D. Docker starts

13. @ExtendWith(MockitoExtension.class) is required because

A. @Mock is not created without that extension
B. It starts PostgreSQL
C. It replaces @Test
D. It writes the coverage file

14. @InjectMocks does which of these?

A. Replaces the service with a mock
B. Builds the service and passes the mocks into the constructor
C. Starts the Spring context
D. Asserts a repository call

15. A stub that forces a collaborator to return 9 proves

A. That 2 + 3 is 5
B. That the caller used the value the collaborator returned
C. That Flyway migrated
D. That the HTTP mapping is correct

16. A mock of PaymentService inside PaymentServiceTest proves

A. The service rule
B. Nothing about the service. The test checks the mock
C. The SQL for save
D. The HTTP status

17. verify(store, never()).save(...) is

A. Arrange
B. An assert that save did not run
C. A skipped test
D. Required on every mock

18. doNothing().when(repository).deleteById(1L) is

A. The assert
B. Arrange for a void method
C. The act
D. A Spring rollback

19. A void method whose promise is "touch nothing" is checked with

A. assertThat(result).isNull()

B. verifyNoInteractions
C. @SpringBootTest
D. MockMvc

20. FeeCalculator.total, a pure function, is tested by

A. Mocking the static method
B. Calling FeeCalculator.total
C. Reflection into a private helper
D. @WebMvcTest alone

21. The default shape for a service that calls a repository is

A. A static method
B. An instance that receives the repository in its constructor
C. A private method tested by reflection
D. A field set with new inside the method

22. Clock.fixed(...) is used so that

A. The suite sleeps until the due date
B. The test chooses the instant
C. Testcontainers freezes the container clock
D. JaCoCo requires a fixed year

23. isAfter(due) when due is one second ahead of the fixed clock returns

A. true
B. false
C. an exception
D. a value that depends on the machine clock

24. isAfter(due) when due equals the fixed clock returns

A. true
B. false
C. an exception
D. null

25. A rule gives 10 percent off when quantity is 5 or more. Four items at 100 cost

A. 450
B. 400
C. 360
D. an exception

26. Under that same rule, five items at 100 cost

A. 500
B. 450
C. 400
D. 0

27. Standard shipping is 2,000 until the amount reaches 50,000, then 0. Shipping on 49,999 for a standard customer is

A. 0
B. 2,000

C. 49,999
D. an exception

28. Gold customers ship free below that line. Shipping on 30,000 for gold is

A. 2,000
B. 0
C. 30,000
D. 500

29. A state machine treats DELIVERED as terminal. Moving DELIVERED to PLACED should

A. Allow the move
B. Throw a conflict
C. Return HTTP 200
D. Leave the status unchanged and return success

30. Login must not reveal which emails exist. A wrong password and an unknown email should

A. Use different messages
B. Share one generic detail
C. Both return HTTP 200
D. Both return HTTP 403

31. A record that must be hidden, rather than described as forbidden, is returned as

A. HTTP 403
B. HTTP 404
C. HTTP 200 with an empty body
D. HTTP 401

32. A @WebMvcTest that stubs the fee method to return 9

A. Proves that 2 + 3 is 5
B. Proves the controller writes the stubbed sum
C. Fails if the fee method is deleted
D. Runs Flyway

33. @MockitoBean belongs on a test that

A. Has no Spring context
B. Has a Spring context and must replace one bean
C. Calls a static method
D. Is a pure unit test of a calculator

34. @DataJpaTest is pointed at PostgreSQL with replace = NONE because

A. H2 runs every PostgreSQL Flyway script unchanged
B. The script is PostgreSQL SQL
C. Redis supplies the datasource
D. MockMvc requires H2

35. @ServiceConnection on a PostgreSQLContainer

A. Copies the JDBC URL by hand in @DynamicPropertySource
B. Registers the container as the application's datasource
C. Starts the React dev server
D. Disables Flyway

36. Static @Container fields on AbstractIntegrationTest

A. Start a new database for every test method
B. Are shared by concrete tests that extend the base
C. Attach to Compose on port 5432
D. Run only when Docker is stopped

37. disabledWithoutDocker = true while Docker is stopped

A. Fails the build
B. Skips the class, and test can still report success
C. Switches the suite to H2
D. Starts the browser

38. @SpringBootTest by itself, with no @Transactional on the test,

A. Rolls every test back
B. Leaves committed rows in place for later tests
C. Drops the schema
D. Starts Chromium

39. @Transactional on a @SpringBootTest method is the wrong tool when

A. You are reading a repository inside @DataJpaTest
B. The behaviour is that a write survives the exception
C. You are asserting a pure function
D. You are naming the test

40. @AfterEach cleanup deletes rows in which order?

A. Parents first
B. Children first, then parents
C. Any order. Foreign keys are ignored in tests.
D. Only the newest row

41. flyway.clean() followed by flyway.migrate()

A. Re-runs every ApplicationRunner automatically
B. Rebuilds the schema. A startup seeder does not run again while the context stays up.
C. Is allowed by Flyway with no extra property
D. Replaces @AfterEach for a pure unit test

42. A message listener that applies a restock after the test publishes an event is waited on with

A. Thread.sleep(5000)
B. Awaitility's await().atMost(...).untilAsserted(...)
C. Clock.fixed
D. A static @Container

43. A real listen port that avoids a server already bound to 8080 is

A. webEnvironment = MOCK
B. webEnvironment = RANDOM_PORT
C. @WebMvcTest
D. @DataJpaTest

44. The purpose of the Playwright journey is to

A. Re-test every fee boundary

B. Prove the full stack from the page to the database
C. Replace the repository tests
D. Raise coverage of private methods

45. Starting a pipeline with TEST_LEVEL=unit

A. Runs ./mvnw verify and the coverage gate
B. Runs the unit job only
C. Starts the Docker daemon
D. Launches Chromium

46. TESTCONTAINERS_HOST_OVERRIDE is set in the CI Docker-in-Docker job because

A. A laptop must use that host too
B. The JDBC URL must reach the daemon service, not localhost inside the Maven container
C. It replaces Flyway
D. The unit job has no other way to mock Redis

47. The JaCoCo rule on verify fails the build when

A. Branch coverage is under 85 percent
B. Line coverage is under 85 percent
C. A mutation survives
D. Instruction coverage is under 100 percent

48. Which fact belongs in FeeCalculatorTest?

A. A card network accepted the charge
B. The dev server kept a header across refresh
C. total(5, 100) is 450 under the ten-percent rule
D. The pay button uses a particular CSS class

49. countAndStore returns 2 and never calls save. Which check goes red?

A. assertThat(n).isEqualTo(2)
B. verify(store).save(text, 2)
C. Both stay green
D. The test does not compile

50. On Spring Boot 4, replacing a bean inside @WebMvcTest uses

A. @MockBean, as older tutorials still show it
B. @MockitoBean
C. @Mock, which also starts the context
D. @InjectMocks

15. Answers

1. C. JUnit calls add. Spring, PostgreSQL, and the browser stay down.

2. B. The level is the boundary. MockMvc never opens a page, so a full Spring context is still integration.

3. C. The browser, the API, and the database are one journey.

4. B. The route guard lives in the frontend. MockMvc never loads that page.

5. B. AssertJ reads the actual value first, then the expected value.

6. C. The three parts are the method, the input, and the result.

7. B. A name that joins two results is two tests.

8. B. Blank and null both return 0.

9. B. Surefire runs *Test, *Tests, and *TestCase. The *IT suffix does nothing by itself.

10. B. -Dgroups=unit selects @Tag("unit"). verify runs every tag and then the coverage gate.

11. B. test writes a report. The check goal runs on verify. A unit-only run leaves controllers unexecuted.

12. B. A tag on the class applies to every test method.

13. A. @Mock stays null until the extension creates it.

14. B. The class under test stays real.

15. B. The stub fixes the return value. The calculator test proves the arithmetic.

16. B. Mock the collaborator. Keep the class under test real.

17. B. never() is the check that a call was required not to happen.

18. B. deleteById returns void. verify is the assert.

19. B. There is no return value. The promise is about the collaborator.

20. B. You own the method. Call it. A static mock replaces the rule.

21. B. Constructor injection is the seam the unit test uses.

22. B. Time becomes an argument. One second is an exact edge.

23. B. The due instant is still in the future. The payment is not overdue.

24. B. isAfter is strict. Equal to now is not after now. That is the edge.

25. B. Four is below the line, so the total is 4 times 100.

26. B. Five takes 10 percent off: 5 * 100 * 90 / 100 = 450.

27. B. 49,999 is still below the free-shipping line.

28. B. The gold condition is free shipping even under the threshold.

29. B. A terminal state refuses the transition. The test asserts the exception.

30. B. Assert the shared detail, not only status 401.

31. B. 404 does not tell the caller that the record exists and is restricted.

32. B. The mock returns 9, so the test stays green if the fee method is deleted.

33. B. Since Boot 3.4, @MockitoBean replaces a bean in the context. @Mock needs no Spring context. Boot 4 keeps @MockitoBean.

34. B. The slice would otherwise substitute an embedded database.

35. B. Boot reads the container. You stop repeating url, username, and password in every test class.

36. B. One pair of containers serves the integration classes. Each class used to start its own pair.

37. B. Read the test count. A skip is not a pass.

38. B. @DataJpaTest rolls back. @SpringBootTest does not, unless you add a strategy.

39. B. The rollback would erase the write the test exists to see. Load the row after the call instead.

40. B. payments, then orders, then customers. The parent delete fails while children still point at it.

41. B. Set spring.flyway.clean-disabled=false. Call the seeder yourself, or keep seed data in a migration.

42. B. Awaitility polls and fails with the assertion. sleep either wastes time or returns too early. Clock.fixed is for a timestamp edge, not for another thread.

43. B. MOCK does not listen. RANDOM_PORT is what the Playwright test navigates toward, directly or through the frontend proxy.

44. B. Fee boundaries stay in unit tests. The journey shows that a person can finish, and that the screen matches the row.

45. B. The unit job's rule matches that value. Integration, end to end, and the gate do not.

46. B. Leave it unset on a laptop, where the daemon is local.

47. B. The gate is bundle line coverage. Branch coverage is reported and is not a gate.

48. C. The gateway in tests is a stub, MockMvc never runs the frontend, and no test looks at CSS. The fee rule is a unit test.

49. B. The number is right, so the assert stays green. The verify is the check that the save happened.

50. B. @MockitoBean has been the context mock since Boot 3.4. Boot 4 keeps it. @Mock is for a test with no Spring context.